

Javascript Scope: Global and Block



CONTENT

01

Global scope

02

Block scope

03

Difference between var , let and const





01

Global Scope

What is Global scope?

Global Scope in JavaScript refers to variables that are accessible from anywhere in your code. A variable declared outside any function or block is considered global and can be used within functions and other blocks.

Declaration
Outside function and curly braces



JS index.js X

JS index.js > ...

```
1  var Name = "Quad Ai";  
2  console.log(Name);  
3  {  
4    console.log(Name);  
5  }  
6  console.log(Name);
```

→ Trying to access

→ Trying to access

→ Trying to access

Output :

```
[Running] node "c:\Users\Lenovo\Desktop\prac\index.js"  
Quad Ai  
Quad Ai  
Quad Ai  
  
[Done] exited with code=0 in 0.118 seconds
```

Note : Getting three outputs as "Quad" which means Name variable is accessible from anywhere.



Use Cases of Global Variables

- Shared Configuration or Constants

Exp `const myCollege= "Quad AI";`

- Access Across Multiple Functions

When multiple functions need access to the same value or o

Exp

```
let isLoggedIn = false;  
function login() {  
  isLoggedIn = true;  
};  
  
function logout() {  
  isLoggedIn = false;  
};
```



Note : use of global variables can lead to conflicts and unintended behaviors in larger applications





02

Block

What is Block scope?

Block Scope in JavaScript refers to variables that are confined within a specific block, such as those defined within curly braces (`{ }`). Variables declared with `'let'` or `'const'` have Block Scope, meaning they are not accessible outside of that block, which helps in managing variable lifetimes and preventing conflicts.

Declaration
Inside function and curly braces




```
JS index.js
JS index.js > ...
1  {
2
3  let Name = "Quad Ai";
4  console.log(Name);
5
6  }
7  console.log(Name);
```

→ Trying to access

→ Trying to access

Output :

```
[Running] node "c:\Users\Lenovo\Desktop\prac\index.js"
Quad Ai
c:\Users\Lenovo\Desktop\prac\index.js:9
console.log(Name);
      ^
ReferenceError: Name is not defined
    at Object.<anonymous> (c:\Users\Lenovo\Desktop\prac\index.js:9:13)
    at Module._compile (node:internal/modules/cjs/loader:1469:14)
    at Module._extensions..js (node:internal/modules/cjs/loader:1548:10)
    at Module.load (node:internal/modules/cjs/loader:1288:32)
```



Note : Got one result and an error





03

**Difference between var ,
let and const**

Hoisting :

Hoisting is the default behavior of JavaScript where **variable and function declarations are moved to the top of their scope** (global or function) **during the compilation phase**

- **Variable declarations (var, let, const)** are hoisted, but:
- var is initialized with undefined.
- let and const are not initialized and stay in the **Temporal Dead Zone (TDZ)** until their declaration

The **Temporal Dead Zone (TDZ)** refers to the period between:
when a let or const variable is **hoisted**, and
when it is **actually declared and initialized** in the code



Difference between var , let and const

Keyword	Scope	Reassignment	Redeclaration	Hoisted
var	Global / Function	✓ Allowed	✓ Allowed	✓ Yes
let	Block	✓ Allowed	✗ Not Allowed	✓ Yes*
const	Block	✗ Not Allowed	✗ Not Allowed	✓ Yes*

